

Table of Contents

1.....	IEEE 802.15.4 & CSS PHY LAYER..	3
1.1	IEEE 802.15.4.....	3
1.1.1	How Low-Cost & Long Battery Life Are Achieved[7]	3
1.1.2	Frequency Band, Data Rate and Channel Numbering	5
1.1.3	Physical Layer responsibilities.....	8
1.1.4	Physical Layer Bit Level.....	8
1.1.5	Physical Layer Services	9
1.1.6	Physical layer packet structure.....	10
1.2	CHIRP SPREAD SPECTRUM (CSS) PHYSICAL LAYER.....	11
1.2.1	Advantages of the CSS (2450 MHz) PHY for LR-WPAN.....	11
1.2.2	Chirp Spread Spectrum Modulation	12
2.....	CSS PHY FLOATING POINT IMPLEMENTATION	18
2.1	CHIRP SPREAD SPECTRUM PHYSICAL LAYER TRANSMITTER....	18
2.1.1	CSS physical layer PDU format.....	18
2.1.2	The PPDU Components:.....	19
2.1.3	PPDU processing:	19
1.	ZERO PADDING.....	20
2.	DEMUX	20
3.	SYMBOL MAPPER.....	20
4.	INTERLEAVER	21
5.	FORM PPDU.....	21
6.	QPSK MAPPER.....	21
7.	DQPSK CODING	21
8.	CSK GENERATION AND MODULATION.....	22
➤	CHIRP SEQUENCE GENERATION	22
3.....	CSS PHY HARDWAR IMPLEMENTATION	24
3.1	CHIRP SPREAD SPECTRUM PHYSICAL LAYER TRANSMITTER....	26
3.1.1	HDL Testbenches.....	30
3.1.2	EDA Simulator Link	31
3.1.3	ChipScope (After Device Programming Verification)	33
REFERENCES.....		35

Acronyms

ACK	Acknowledgment
AM	Amplitude Modulation
AWGN	Additive White Gaussian Noise
BER	Bit Error Rate
CCA	Clear Channel Assessment
CLB	Configurable Logic Blocks
CORDIC	Coordinate Rotation Digital Computer
CSK	Chirp Sequence Generation
CSS	Chirp Spread Spectrum
ED	Energy Detection
EDA	Electronic design automation
FFD	Full Function Device
FM	Frequency Modulation
FPGA	Field Programmable Gate Array
HAN	Human Area Network
HDL	Hardware Design Language
IR	Infrared
LQI	Link Quality Indicator
LR-W	Low Rate Wireless
MAC	Medium Access Control
MDMA	Multiple Dimensional Multiple Access
NCD	Native Circuit Description
NGD	Native Generic Database
PAN	Personal Area Network
PER	Packet Error Rate
PHR	PHY Header
PM	Phase Modulation
PPM	Part Per Million
PSDU	PHY Service Data Unit
RFD	Reduced Function Device
SFD	Start Frame Delimiter
SNR	Signal to Noise Ratio
SoC	System on Chip
WSN	Wireless Area Networks

IEEE 802.15.4 & CSS PHY Layer

The IEEE Std 802.14.5 is a multi-optional communication MAC and PHY layer with a medium-size set of primitives that can support a large variety of higher layer protocols. This chapter presents an overview on the IEEE 802.15.4. It particularly focuses on the IEEE 802.15.4 physical layers, which are the most relevant in the Thesis context. It also provides a detailed prelude on Chirp Spread Spectrum (CSS) physical layer and its advantages, as it is the main interest of this Thesis.

1.1 IEEE 802.15.4

In order to control the administrative costs of both implementer and user IEEE Std 802.15.4 is designed to be a short-range service without infrastructure, and able to support very large networks so both transmitter and receiver should consume little power and have a low operating cost.

1.1.1 How Low-Cost & Long Battery Life Are Achieved[7]

Duty cycle

The battery is a relatively high-cost component in a transceiver system. The battery provides the energy for the communications, but at a cost of operation and replacement. To achieve long battery life, the energy must be taken continuously at an extremely low-rate, or in small amounts at a low-duty cycle, IEEE Std 802.15.4 was designed to support very-low duty-cycle operation. The standard allows some device, to operate at low-duty cycles with both the transmitter and receiver inactive for over 99% of the time they are in operation.

However, in practical implementations there will always be a small amount of standby power consumed for timers, when the wireless circuits are inactive. To reduce the time-average power consumption, both the active power and the standby power of an implementation should be reduced as much as possible. For most applications, the active power is much larger than the standby power. Under this assumption, we can see that by reducing the duty cycle, low-power consumption levels, associated long battery lifetimes can be reached.

To support low-duty cycles, the IEEE Std 802.15.4 beacon packet can be as short as 544 μ s in the 2.4 GHz band, while the super-frame period (the time between network beacons) may be extended from 15.36 ms to over four minutes. This results in a beacon duty cycle that may be set from 2.3% to 0.000216% (2.16 ppm). Further, the standard supports a non-beacon mode in which a network may operate without any beacons. The non-beacon mode enables the slaves in a master-slave star network, for example, to remain in standby mode indefinitely, only contacting the master (which may be mains powered, perhaps, and therefore capable of constant reception) when an event occurs. The slaves may therefore have an almost unlimited battery life, limited primarily by their standby power consumption.

In order to minimize device duty cycle, The IEEE Std 802.15.4 supports relative high data rates.

For example, for a device with 10 mW active power and 10 μ W standby power, and the duty cycle is 0.1 %, then the time-average power drain is about 19.99 μ W($10,000 \mu\text{W} * 0.1 \% + 10 \mu\text{W} * 99.9 \%$). If the device is supplied by a 750 mAh AAA battery, linearly regulated to 1 V, it will have a battery life of more than 37.000 hour" or more than four years.

Modulation

With its low-cost and low-power consumption goals in mind, the IEEE Std 802.15.4 communications protocol was designed to support digital data communication only (i.e., analog service is not supported). Data-only service allows the modulation scheme to be chosen to be highly efficient, enabling a low-cost implementation. Likewise, the protocol supports only half-duplex operation so that the transmitter and receiver are not required to be active simultaneously.

IEEE Std 802.15.4 uses a quasi-orthogonal modulation scheme, where each symbol is represented by one of N nearly orthogonal PN sequences, specified by each PHY layer. This is a power-efficient modulation method that achieves low signal-to-noise ratio (SNR) and signal-to-interference ratio (SIR) requirements at the expense of a signal bandwidth. A typical low-cost detector implementation is expected to meet the 1% PER requirement as SNR values of 5-6 dB.

Further, the data modulation schemes are simple raised-cosine-shaped Phase Shift Keying (PSK) or half-sine-shaped Offset Quadrature Phase Shift Keying (O-QPSK) in the 2.4 GHz PHY. Each of these forms of modulation maintains a peak-to-average carrier power ratio of one, which minimizes both power consumption and implementation complexity.[7]

Transmitter power

IEEE Std 802.15.4 requires only the compliant device be capable of transmitting -3 dBm within the instantaneous power capacity of inexpensive battery sources and within the capability of highly integrated and low-cost System-on-Chip (SoC) implementations.[7]

Receiver sensitivity

IEEE Std 802.15.4 requires minimum -92 dBm or -85 dBm sensitivity due to different PHYs. This specification permits the use of a simple receiver, low-cost receiver designs, with little radio-frequency amplification (which reduces power consumption).

Quality of service

To reduce implementation complexity, IEEE Std 802.15.4 employs a simple full-handshake protocol to ensure reliable data transfer and good QoS and it does not support isochronous communication and it does not support multiple classes of service within a single PAN. An exception is that the standard does support the use of Guaranteed Time Slots (GTS), which reserve network time for synchronous communication (avoiding potential channel access delays), at the discretion of the PAN coordinator. This extends the application space of IEEE Std 802.15.4 devices to include applications requiring low-latency communications, such as wireless Joy sticks

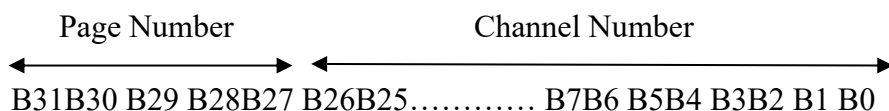
and mice.

1.1.2 Frequency Band, Data Rate and Channel Numbering

The frequency band and data rate depend on a country's regulations and standards. In the European countries, the European Standard Institute recommends the frequency band of 868 to 868.6MHz. Whereas in the countries of North America, Australia, and New Zealand, the FCC recommends the 915MHz and 2.4GHz bands.

In its 2009 revision, IEEE 802.15.4 added physical layers support new frequency bands in China and Japan. **Table 1-1** shows the IEEE 802.15.4 frequency bands, frequency ranges and country. **Table 1-2** shows the IEEE 802.15.4 frequency bands, frequency ranges, data rates and modulation.

IEEE 802.15.4 uses 32 bits to represent the channel number. This 32-bit number is divided into two segments, as shown here



The leftmost 5 bits—B31B30B29B28B27—represent the page number. This allows for the designation of 32 pages. Each bit of B26B25... B7B6 B5B4 B3B2 B1B0 represents a channel number within the page. Because the physical layer of a device can support multiple channels within a page, each channel must be represented by a distinct bit. Table 1-3 describes channel number and channel page relationship while Table 1-4 shows an example for channel assignment describing page 0.

Table 1-1, IEEE 802.15.4 frequency bands, frequency ranges and country

Frequency Band	Frequency Range	Region
780 MHz	779–787 MHz	China
868 MHz	868–868.6 MHz	North America, Australia, and New Zealand
915 MHz	902–928 MHz	European countries
950 MHz	950–956 MHz	Japan
2.4GHz	2400–2483.5 MHz	World Wide

Table 1-2, IEEE 802.15.4 frequency bands, frequency ranges, data rates and modulation

Frequency Band	Frequency Range	Modulation	Data Rate
868 MHz	868–868.6 MHz	BPSK	20Kbps
868 MHz optional		O-QPSK	100Kbps

868 MHz optional		ASK	250Kbps
915 MHz	902–928 MHz	BPSK	40Kbps
915 MHz optional		O-QPSK	250Kbps
915 MHz optional		ASK	250Kbps
950 MHz	950–956 MHz	BPSK	20Kbps
950 MHz optional		GFSK	100Kbps
780 MHz optional	779–787 MHz	MPSK	250Kbps
780 MHz optional		O-QPSK	250Kbps
2.4GHz	2400–2483.5 MHz	O-QPSK	250Kbps

Table 1-3, IEEE 802.15.4 page number, frequency band, modulation and channel number

Channel Page	Channel Number	Channel Description
0	0	Channel 0 is in 868 MHz band using BPSK
	1-10	Channels 1 to 10 are in 915 MHz band using BPSK
	11-26	Channels 11 to 26 are in 2.4 GHz band using O-QPSK
1	0	0 Channel 0 is in 868 MHz band using ASK
	1-10	Channels 1 to 10 are in 915 MHz band using ASK
	11-26	Reserved
2	0	Channel 0 is in 868 MHz band using O-QPSK
	1-10	Channels 1 to 10 are in 915 MHz band using O-QPSK
	11-26	Reserved
3	0-13	Channels for CSS PHY
4	0	Channel 0 is sub-gigahertz band for UWB PHY
	1-4	1 to 4 are low band for UWB PHY
	5-15	5 to 15 are high band for UWB PHY
5	0-3	Channels 0 to 3 are in 780 MHz band using O-QPSK
	4-7	Channels 4 to 7 are in the 780 MHz band using MPSK
6	0-9	Channels 0 to 9 are in 950 MHz band using BPSK
	10-21	Channels 10 to 21 are in 950 MHz band using GFSK 5–15 Channels
7-31	Reserved	Reserved

Table 1-4, IEEE 802.15.4 page 0 channel assignment

Frequency Band	Channel Number	Center Frequency
868 MHz	0	868.3
915 MHz	1	906
	2	908
	3	910
	4	912
	5	914
	6	916
	7	918
	8	920
	9	922
	10	924
2.4GHz	11	2405
	12	2410
	13	2415
	14	2420
	15	2425
	16	2430
	17	2435
	18	2440
	19	2445
	20	2450
	21	2455
	22	2460
	23	2465
	24	2470
	25	2475
	26	2480

1.1.3 Physical Layer responsibilities

The physical layer (PHY) provides the interface with the physical medium where the actual communications occur. The PHY layer is the lowest component in the ISO/OSI model and is in charge of:

- Activation and deactivation of the radio transceiver
- Energy detection (ED) within the current channel
- Link quality indicator (LQI) for received packets
- Clear channel assessment (CCA) for carrier sense multiple access with collision avoidance (CSMA-CA)
- Channel frequency selection
- Data transmission and reception

1.1.4 Physical Layer Bit Level

PHY layer bit level processing consists of assembling number of bits into a symbol, converting that symbol to a chip sequence using Tables in [4], each table related to specified PHY layer , and modulating that chip sequence on the I or Q channel, respectively. The process is diagrammed in **Figure 1-1**. **Table 1-2** shows the IEEE 802.15.4 frequency bands, frequency ranges, data rates and modulation.

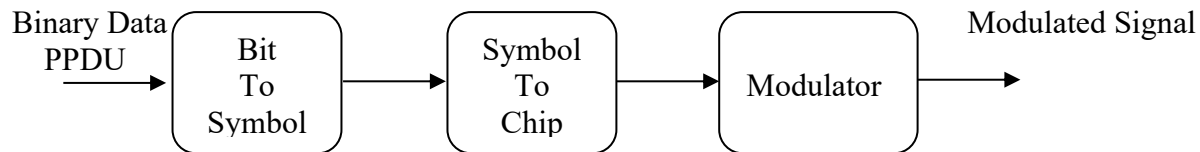


Figure 1-1: PHY layer bit level processing.

1.1.5 Physical Layer Services

The PHY layer provides an interface between the physical radio channel and the MAC sub-layer through the use of two services. These services are the PHY data service and the PHY management service.

- PHY Data Services

The PHY data service provides primitives to the MAC sub-layer. **Figure 1-2** shows scenario of message sequence exchange.

1. Request: The request primitive is passed from the calling layer (also called the user layer) to request that a service is initiated.
2. Indication: The indication primitive is passed from the service layer to the user layer to indicate an internal event of significance. This event may be logically related to a remote service request, or it may be caused by a service layer internal event.
3. Response: The response primitive is passed from the user layer to the service layer to complete a procedure previously invoked by an indication primitive.
4. Confirm: The confirm primitive is passed from the service layer to the user layer to convey the results of one or more associated previous service requests.

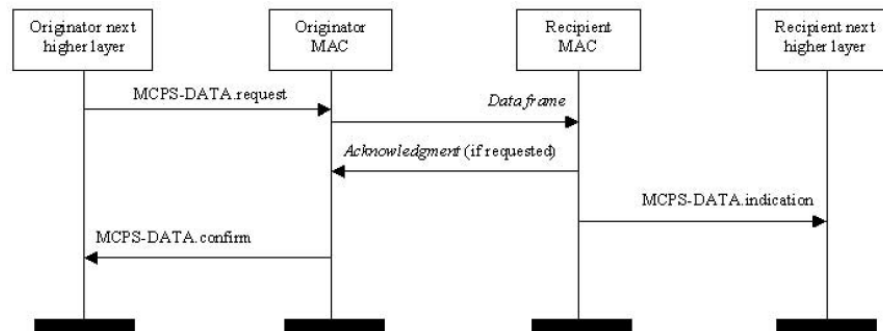


Figure 1-2: Message sequence diagram for data packet exchange mechanism.

- PHY Management Services

The PHY management service provides support for commands to control communication settings and the radio control functionality. The PLME (PHY layer management entity) primitives are summarized in **Table 1-5**:

Table 1-5, IEEE 802.15.4 PHY Management Services primitives

Primitive	Category	Purpose
GET	PHY PAN Information base management	To manage the PHY layer: Read process
SET		To manage the PHY layer: write process
SET_TRX_STATE	Enable/Disable radio system	control the radio transceiver and enable lower power consumption
CCA	RF Sensing: Clear Channel Assessment	When the clear channel assessment is requested, the PHY layer enables the Receiver performs a CCA measurement and then disables the receiver. Once the clear channel assessment measurement is completed, the PHY layer confirms indicating if the channel is busy or not.
ED	RF Sensing: Energy Detection	The ED primitive allows a device to perform RF energy detection in the actual channel where it is operating. The measurement performed is similar to the PLME -CCA, but with higher resolution, it returns an energy level in range from 0 10 255. The use of this primitive can enhance the functionality often network layers.

1.1.6 Physical layer packet structure

The PHY Protocol Data Unit (PPDU) is the packet data structure at the PHY protocol level that modulates the wireless transmitter. The PPDU consists of three components:

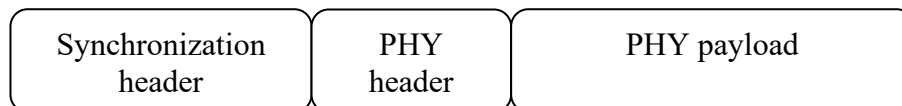


Figure 1-3: Physical layer packet structure

- Synchronization header

Synchronization header consists of two fields a preamble and a start-of-frame delimiter (SFD). The preamble consists of 32 or 80 bits, PHY dependence. The preamble field allows a receiver to achieve chip synchronization and bit synchronization. The SFD allows the receiver to find the beginning of the frame.

- PHY header

The PHY header indicates the length of the payload in bytes. The PHR format is shown below

Bit 0 Bit 6	Bit 7 Bit 8	Bit 9 Bit 10 Bit 11
Length of payload (bytes)	Not used	Reserved

- PHY payload

The PHY payload is composed of only one field called the Physical layer Service Data Unit (PSDU)..

1.2 Chirp Spread Spectrum (CSS) Physical Layer

In its 2007 revision, IEEE 802.15.4 added Chirp Spread Spectrum (CSS) alternative physical layer (PHY) operating in the 2450 MHz band.

CSS is a spread spectrum system that is similar to direct sequence spread spectrum (DSSS) and offers some significant properties in addition to these systems due to the different modulation methods. A chirp is a linear frequency modulated pulse. It could be thought of as sweeping the band at a very high speed. The type of CSS system defined for this standard uses patterns of smaller chirps, or “subchirps”, to build one larger chirp symbol. This allows multiple networks to use the same frequency channel simultaneously and also offers more robust performance. This CSS system also uses differential quadrature phase-shift keying (DQPSK) modulation to further enhance performance.

1.2.1 Advantages of the CSS (2450 MHz) PHY for LR-WPAN

The CSS LR-WPAN specification is designed to provide robust performance for LR-WPAN applications while leveraging the unique capability of CSS waveforms to support long-range links or to support links to mobile devices moving at higher speeds. The CSS PHY is intended to take advantage of the global deploy ability of the 2450 MHz band due to favorable regulations, while offering enhanced robustness, range, and mobility. In addition CSS give LR-WPAN devices enhanced immunity to multipath fading and extended range for robust performance with very low transmit power.

Two data rates are specified for CSS in order to offer implementers the flexibility to select the

rate and properties best suited for their applications, as the following guidelines illustrate:

- The lower/coded rate would be appropriate in quiet additive white Gaussian noise (AWGN) and high multipath environments.
- The higher rate would be appropriate for low-energy consumption and burst interference environments.

1.2.2 Chirp Spread Spectrum Modulation

CSS was developed by Nanotron [9] in 2007. CSS comes as an ideal communication method by an extremely elegant combination of the three modulation schemes: frequency, amplitude, and phase modulation. This section gives a brief introduction about CSS.

The history of communication engineering shows a permanent evaluation of modulation schemes to meet the requirements of upcoming applications. The first technique developed was Amplitude Modulation or AM, but it is very sensitive to noise, so a second technique was developed to reduce this sensitivity. This is Frequency Modulation or FM, but it requires almost twice as much bandwidth as AM. With the advent of the digital age, Phase Modulation or PM was introduced to significantly reduce the required bandwidth as compared to that required by FM. But it too has to fight against disturbances and fluctuation of the carrier frequency.

2.1.2.1 Modulation Scheme Properties

Table 1-6 provides the properties of Amplitude Modulation (AM), AM Single Side Band modulation (SSB), Frequency Modulation (FM), and Phase Modulation (PM).

Table 1-6, modulation scheme properties

Modulation	bandwidth B in the RF range for a bandwidth b in the baseband	Properties
Amplitude Modulation (AM)	$B = 2b$	SNR dependent on amplitude.
AM Single Side Band modulation (SSB)	$B = 1b$	Complicated due to the correct phase adjustment of the carrier in the RX; no IQ modulation
Frequency Modulation (FM)	$B = 4b$	Resistant, but requires large bandwidth
Phase Modulation (PM)	$B = 2b$	Susceptible to phase noise and other disturbances

2.1.2.2 MDMA: The Best of AM, FM and PM

None AM, FM, or PM is ideal as the benefits of each scheme are offset by its deficiencies. However, it is possible to build an ideal system that transmits information with a predetermined quality without wasting precious bandwidth. This can be done by intelligently combining these three modulation techniques into a single method Multi Dimensional Multiple Access (MDMA).

This method combines the best from each scheme in such a way that the sum becomes greater than its constituent parts.

➤ The Development of MDMA

Two basic considerations led to the development of MDMA:

1. Data should be transmitted from the transmitter to the receiver by the modulation of the carrier frequency (information). Until now this task was taken by one of the three modulation schemes, either FM, AM, or PM, but each of these schemes demonstrates specific advantages and disadvantages.
2. The transmitted symbols must be adapted to their individual requirements and transmission conditions (transportation). Initially, some general misunderstandings needed to be removed. For instance, the range of a signal is not dependent on its power; but rather a function of the energy supplied from the transmitter that enables the signal to reach the receiver. As an analogy, the distance a car can travel is dependent on the amount of fuel (energy) that is available to run the car's engine, and is completely independent from the horsepower of the engine. Thus, the main objective of communications engineering must be to apply exactly the correct amount of energy to each symbol to secure a successful transmission (which is based on the maximum acceptable Bit Error Rate or BER) without wasting precious bandwidth.

➤ Sinc Pulses and Chirp Pulses

MDMA uses two elementary signals for the processing and transmitting of symbols Sinc pulses and Chirp Pulses. Both of these signals have partly complementary properties and have a special feature.

- Sinc Pulses

For baseband processing in the transmitter and receiver, ideal sinc pulses are deployed. These pulses have the shortest possible duration at a given bandwidth B . Through this, the sequence of the bits in the baseband is optimal. Furthermore, sinc pulses are generated relatively easily in the transmitter and can be detected with simple amplitude discrimination in the receiver.

Figure 1-4 and **Figure 1-5** show the sinc pulse in the time domain and frequency domain respectively.

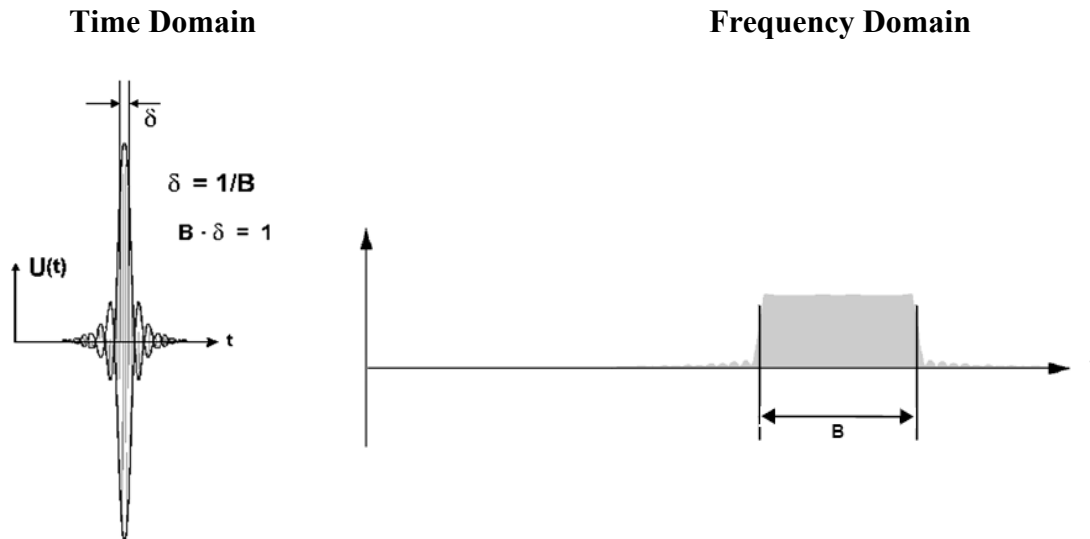


Figure 1-4: Sinc pulse in the time domain

Figure 1-5: The spectrum of the sinc pulse in the frequency domain

- Chirp Pulses - Linear Frequency Modulated Signals

MDMA uses chirp pulses for transmission over air. Chirp pulses are Linear Frequency Modulated (LFM) signals with constant amplitude. They fill out the total available bandwidth B over a predefined duration T . A time-bandwidth product (BT product) can be realized that is much larger than 1. The larger is the time-bandwidth product, the more resistant the chirp pulses are against disturbances during transmission. **Error! Reference source not found.** and **Error! Reference source not found.** illustrate a chirp pulse (LFM pulse) in the time domain and frequency domain respectively.

The same spectrum is not the only similarity of chirp and sinc pulses. These pulses can be simply and reversibly transformed from one to another with a Dispersive Delay Line (DDL). The combination of these two signals form the basic principle of MDMA.

2.1.2.3 AM, FM, and PM Contribution to MDMA

The expression of the chirp pulse, which is distributed over the transmission channel, shows an integration of all three modulation schemes as shown in **Table 1-7**. The signal energy is defined as U_0 with the Amplitude Modulation. Each symbol uses the total available bandwidth B by Frequency Modulation. The variable phase is utilized for the transmission of the information. Each of the three modulation schemes makes a specific contribution to MDMA.

Table 1-7, mathematical expressions of MDMA signals

Pulse	Expression
Sinc pulse (baseband)	$U(t) = U_0 \frac{\sin(\pi B t)}{\pi B t}$
Sinc pulse (RF band)	$U(t) = U_0 \frac{\sin(\pi B t)}{\pi B t} \cos(w_0 t + \varphi)$
Chirp pulse	$U(t) = \underbrace{\frac{U_0}{\sqrt{Bt}}}_{\text{AM}} \cos\left(\underbrace{w_0 t}_{\text{FM}} + \underbrace{\frac{ut^2}{2}}_{\text{PM}} + \varphi\right)$

- Contribution of Amplitude Modulation: Symbol Energy

The energy of the transmitted symbol is defined by the amplitude U_0 . The amplitude can be adapted easily to the requirements for the transmission of the information (such as range and BER) during operation at the transmitter. Thus, a higher BER for voice transmission can be accepted (using lower amplitude) than that required for important data (using higher amplitude).

- Contribution of Frequency Modulation: Robustness

The modulation of the frequency guarantees robustness against disturbances:

- Amplitude fills the total available bandwidth: frequency spreading

The LFM pulse with constant amplitude fills the total available RF bandwidth B . Thus, MDMA always uses the total available bandwidth B of the channel for the transmission of information, even if the data rate does not require the full bandwidth. This technology is known as Frequency Spreading. The spread factor (ν) is defined as bandwidth B divided by data rate R ($\nu = B / R$). Narrowband noise does not affect the transmission of the symbol significantly because the full bandwidth of the RF channel is used. Enough uninfluenced symbol energy remains to reach the receiver.

- Signal passes the full bandwidth: time spreading

The second parameter that can be set is the duration T in which the signal passes the full bandwidth (duration T of the chirp pulse). This technology is known as Time Spreading. This is the duration T of the chirp pulse divided by shortest symbol duration: ($\Psi = T/\delta = BT$).

The effect of time and frequency spreading is a high time-bandwidth product, consequently, the

signal has a high resistance against disturbances (noise).

- Contribution of Phase Modulation

The actual information can be embedded in the signal using the phase. This becomes one of the major advantages of MDMA; the amplitude of the signal in the transmission channel is almost constant over time.

2.1.2.4 CSS Pros

- Robust against Noise and Multipath Fading due to high BT Product
- Most effective Utilization of the given Bandwidth
- Simple Synchronization due to chirp pattern repetition
- Low power consumption
- Quality on Demand support

The amplitude of the chirp pulses can be set to its permitted maximum in order to meet the extreme requirements of quality or range. The amplitude can be reduced for lower requirements by superposing many chirp pulses to increase the data rate.

2.1.2.5 CSS Cons

- Complexity
- Hardware Implementation

This Page Intentionally Left Blank

CSS PHY Floating Point Implementation

The 2007 amendment of the IEEE 802.15.4 standard presented Chirp Spread Spectrum (CSS) PHY as a new alternative physical layer. Spread Spectrum (SS) is a technology where the spectrum of the signal is increased over the minimum required to transmit the data to mitigate the effect interference. This is important in the WPAN systems, working in unlicensed bands, since several systems can use the same band without any coordination. Chapter 3 discusses Chirp Spread Spectrum (CSS) modulation in details.

The CSS PHY supports 2 data rates, namely, 1 Mbps and 250 kbps. Most of the PHY processing is the same for the two data rates. However, there are some differences which will be indicated in the corresponding places below.

The aim of this Thesis is to design and implement a practical transmitter and receiver for the Chirp Spread Spectrum (CSS) physical layer of the IEEE 802.15.4 standard. This implementation should comply with the requirements given in the standard. This chapter describes the developed floating-point model for transmitter and receiver. Also it shows receiver performance in different conditions.

2.1 Chirp Spread Spectrum Physical Layer Transmitter

IEEE 802.15.4 Std [8] describes CSS PHY transmitter. This section presents CSS PHY packet structure and CSS PHY transmitter functions in order of processing time.

2.1.1 CSS physical layer PDU format

In the transmission side the MAC delivers to the PHY bytes to be transmitted, which are called the PHY service data unit (PSDU). The PHY encapsulates the PSDU in what is called the PHY protocol data unit (PPDU). The PPDU format of the CSS PHY is given in **Table 2-1** for both data rates. The “symbol” in this figure is the CSS Chirp Sequence that will be explained later. The PPDU starts with a preamble that will be used by the receiver for time and frequency synchronization and some other tasks. The preamble is followed by a Start Frame Delimiter (SFD) which declares the end of the preamble and the start of the PHY header (PHR) and the PSDU payload. The SFD also indicates which data rate is being used in this PPDU. The PHR mainly provides information to the receiver about the length of payload in unit of bytes.

Table 2-1, Format of the CSS PPDU

Data rate	SHR		PHR	PSDU
	Preamble	SFD		
1 Mbps	8 symbols	4 symbols	2 symbols	variable
250 kbps	20 symbols	4 symbols	8 symbols	variable

2.1.2 The PPDU Components:

➤ Preamble

For the 1 Mbps data rate, the preamble consists of 32 ones, while for the 250 kbps it consists of 80 ones.

➤ SFD

The SFD field consists of 16 bits which are different of the 1 Mbps and 250 kbps data rates. The SFD field is shown below:

Data Rate	Bit (0:15)															
1 Mbps	-1	1	1	1	-1	1	-1	-1	1	-1	-1	1	1	1	-1	-1
250 kbps	-1	1	1	1	1	-1	1	-1	-1	-1	1	-1	-1	-1	1	1

➤ PHR

The PHR indicates the length of the payload in bytes. The PHR format is shown below:

Bit 0 Bit 6	Bit 7 Bit 8	Bit 9 Bit 10 Bit 11
Length of payload (bytes)	Not used	reserved

➤ PSDU

The PSDU consists of the bytes delivered from MAC to PHY to be transmitted wirelessly. The PSDU has at most 127 bytes.

2.1.3 PPDU processing:

The PPDU is processed according to **Figure 2-1**. The processing steps are explained in details in the standard and are briefly presented below.

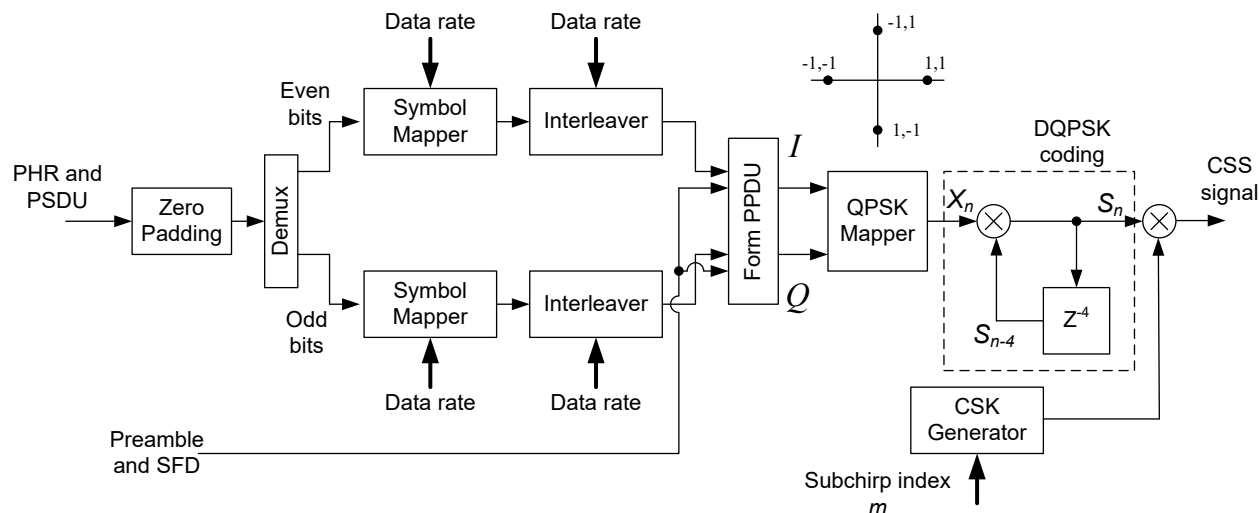


Figure 2-1, Generation of the CSS signal

The preamble and SFD are generated as explained above. The PHR is also generated and followed by the payload. The PHR and payload are padded with zeros to make their total number of bits a multiple of 6 or 24 for data rates 1 Mbps and 250 kbps, respectively. The PHR and payload are de-multiplexed into I/Q paths as shown in **Figure 2-1**. The processing blocks of **Figure 2-1** are described below:

1. Zero Padding

This block pads zero bits at the end of the payload to make the total number of bits in the PHR and PSDU a multiple of N , where $N=6$ for data rate 1 Mbps and $N=24$ for 250 kbps. This is to make sure that the *Symbol Mapper* and *Interleaver* blocks have sufficient bits to process.

2. Demux

Even numbered bits are forwarded to the I path (upper part) and odd bits are forwarded to the Q path (lower part).

3. Symbol Mapper

This block converts each sequence of n bits into a codeword of m bits. The values of n and m are given below.

This corresponds to block coding of rates $3/4$ and $3/16$ for data rates 1 Mbps and 250 kbps, respectively. The list of 8 code words for data rate 1 Mbps and 64 code words for 250 kbps are given in tables 26a and 26b in the standard[8].

Data rate	n	m
1 Mbps	3	4
250 kbps	6	32

In the 1 Mbps case the first 4 code words are Walsh-Hadamard codes of size 4, while the remaining 4 code words are the same with negated sign. In the 250 kbps case, the first 32 code

words are Walsh-Hadamard codes of size 32, while the remaining 32 code words are the same with negated sign.

4. Interleaver

The Interleaver size is 64 bits and it is used only with the 250 kbps data rate. There is no Interleaver for the 1 Mbps data rate. The interleaving process can be described as follows. The bits to be interleaved are divided into 16 groups, G_0 to G_{15} , with 4 bits each. Hence, G_n includes bits from b_{4n} to b_{4n+3} . These groups are interleaved as follows.

Interleaver input															
G0	G1	G2	G3	G4	G5	G6	G7	G8	G9	G10	G11	G12	G13	G14	G15

Interleaver output															
G0	G13	G2	G15	G4	G9	G6	G11	G8	G5	G10	G7	G12	G1	G14	G3

5. Form PPDU

In this block the PPDU is formed from the preamble, SFD, PHR and zero-padded payload according to **Figure 2-1**.

6. QPSK Mapper

I and Q bits are mapped to QPSK symbols as shown in the constellation of **Figure 2-1**

7. DQPSK Coding

The differential QPSK coding block multiplies each QPSK symbol with the output delayed by four symbols. If the output of the QPSK Mapper is x_n , then the output S_n of the DQPSK Coding block is given by

$$S_n = X_n S_{n-4} \quad (1)$$

The initial value of the first 4 DQPSK symbols is $S_0 = S_1 = S_2 = S_3 = \exp(j\pi/4)$.

8. CSK Generation and Modulation

This is the key process in the CSS transmitter. This CSK Generation block generates one of 4 possible Chirp Sequences according to the desired index m which takes values $m=1, 2, 3$ or 4 . A chirp sequence itself consists of 4 subchirps followed by a time gap. The 4 subchirps are indexed as $k=0, 1, 2$ and 3 . A subchirp is a sinusoidal waveform with increasing or decreasing frequency. In what follows we will assume a sampling frequency $f_s=32$ MHz.

Each Chirp Sequence includes 4 Subchirps with duration $1.1875 \mu s$ each, which is equivalent to 38 samples. Each Subchirp is multiplied by one of the DQPSK symbols S_n , as shown in **Figure 2-2**. Hence, a Chirp Sequence is modulated by 4 successive DQPSK symbols S_n, S_{n+1}, S_{n+2} , and S_{n+3} . Obviously, the index n in **Figure 2-3** is always a multiple of 4. The group of 4 subchirps is followed by a time gap, T_{even} or T_{odd} . This time gap is different for even and odd Chirp Sequences. The sum of the even and odd gaps is always $2.5 \mu s$, which is equivalent to $T_{even} + T_{odd} = 80$ samples. Hence, a Chirp Sequence consists of 4 Subchirps and a time gap. **Table 2-2** gives the values of the time gaps for even and odd Chirp Sequences.

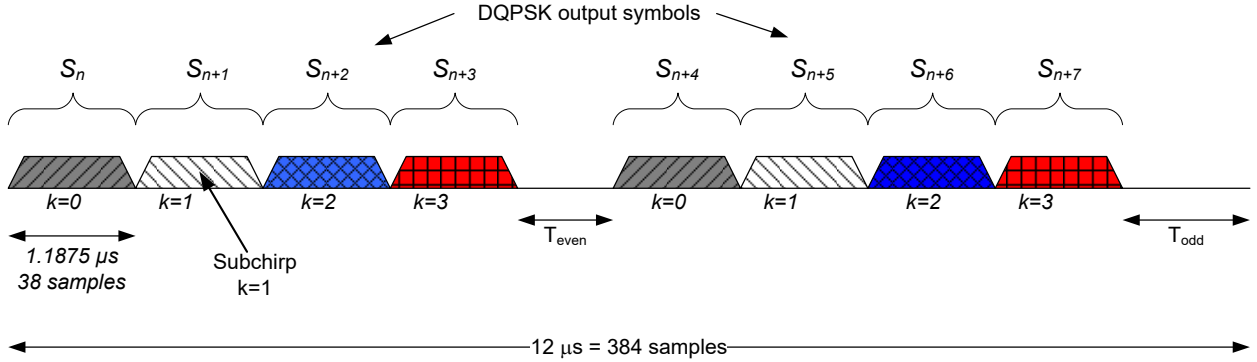


Figure 2-2, Chirp sequence durations

Table 2-2, Values of Time gaps for even and odd chirp sequences with different index m

Chirp Sequence Index m	T_{even} (samples)	T_{odd} (samples)
1	10	70
2	20	60
3	30	50
4	40	40

➤ Chirp Sequence Generation

A Chirp Sequence with index m includes 4 subchirps with index $k=0, 1, 2$ and 3 . Each subchirp is generated according to the following equation. We use Matlab notation for simplicity. Let array equal to $(-19:18)$. Then the 38 samples of $Subchirp_{m,k}$ are given by:

$$Subchirp_{m,k}(0:37) = \exp \left[j 2\pi \left(\frac{f_{k,m}}{f_s} + \frac{\mu \zeta_{k,m}}{2f_s^2} * array \right) .* array \right] .* RaisedCosine \quad (2)$$

The parameters in the previous equation are given by: $\mu=7.3158 \text{ s}^{-2}$, $f_s=32 \text{ MHz}$, $f_{k,m} = \pm 3.15 \text{ MHz}$ and the sign of $f_{k,m}$ is given in the **Table 2-3**. Also, $\xi_{k,m} = \pm 1$ is according to **Table 2-4**.

Table 2-3, Sign of $f_{k,m}$

Sign of $f_{k,m}$				
$m \backslash k$	0	1	2	3
1	-	+	+	-
2	+	-	-	+
3	-	+	+	-
4	+	-	-	+

Table 2-4, Value of $\xi_{k,m}$

Value of $\xi_{k,m}$				
$m \backslash k$	0	1	2	3
1	+1	+1	-1	-1
2	+1	-1	+1	-1
3	-1	-1	+1	+1
4	-1	+1	-1	+1

Note that the value $f_{k,m}$ is the centre frequency of the subchirp, while the value $\xi_{k,m}$ indicates whether the frequency of this subchirp is increasing (+1) or decreasing (-1). Finally, RaisedCosine is the raised cosine function with roll-off factor $\alpha = 0.25$. The positive index part ($n=0$ to $n=19$) of the function is given by:

$$\text{RaisedCosine} = \begin{cases} 1 & 0 \leq n \leq 11 \\ \frac{1}{2} \left[1 + \cos \left(\frac{5\pi}{38} (n - 0.3 \cdot 38) \right) \right] & 12 \leq n \leq 19 \\ 0 & n > 19 \end{cases} \quad (3)$$

The negative index part is the mirror image of the positive index part. **Figure 2-3** shows the raised cosine function used in the CSS generation.

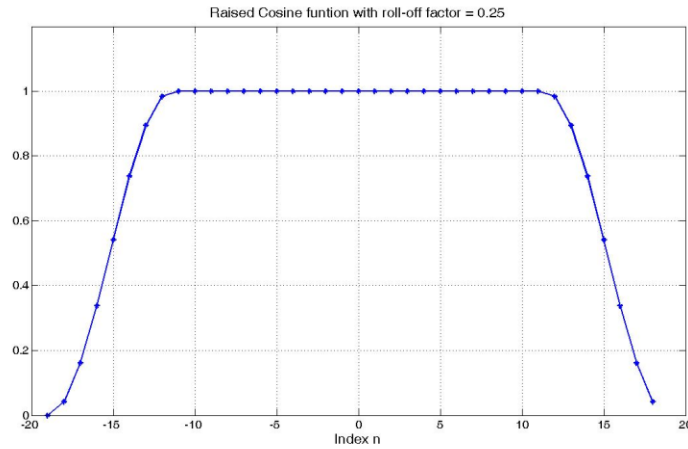


Figure 2-3, Raised cosine function used in CSS

CSS PHY Hardwar Implementation

This thesis goal is FPGA Implementation of Chirp Spread Spectrum ZigBee Physical Layer transceiver. Our target is Xilinx Spartan-3a board[12]. Design flow chart is shown in Figure 3-1. Design flow goes as follows:

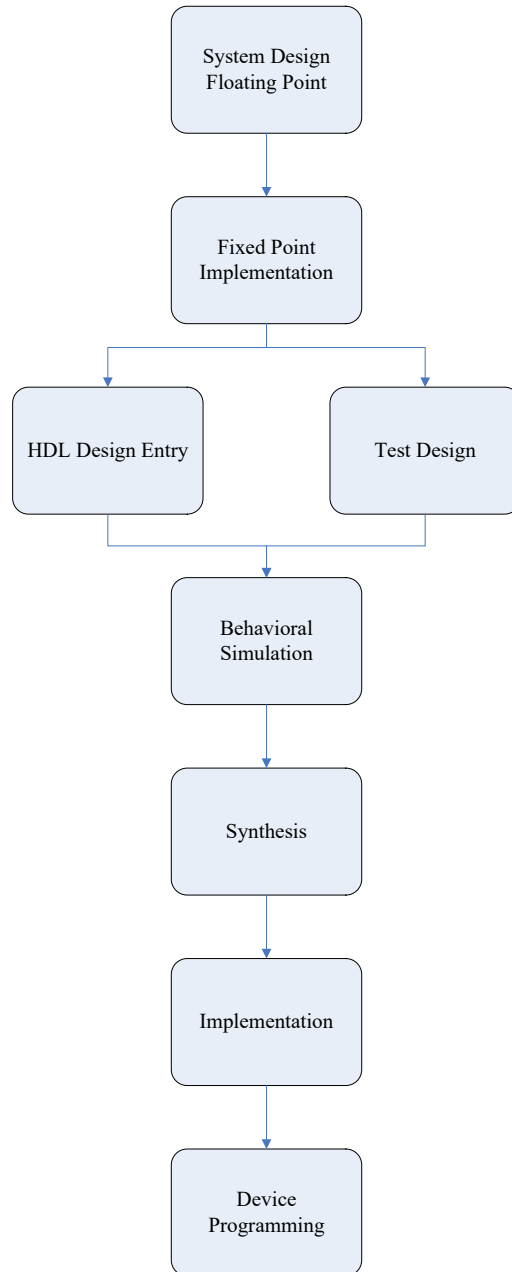


Figure 3-1, Design flow

- **System Design:**

This stage involves analysis of the requirements, problem decomposition and functional simulation. In this stage we create a floating point model for CSS PHY. The model implementation and performance were presented in previous chapter.

Tool: Matlab

- **Fixed Point Implementation**

This is the entry stage to digital design. In this stage we convert floating point model to fixed point model, which specify number of bits for all variables and fix all process, and adjust fixed point performance to match the requirements.

Tool: Matlab, fi toolbox.

- **Design Entry**

There are different techniques for design entry, Schematic based and Hardware Description Language (HDL). There are different design representations (Behavioral, Structural, Dataflow and Mixed) descriptions. In our implementation we use HDL (VHDL and Verilog) and (Structural and Behavioral) descriptions.

Tool: HDL Editor, Notepad++ & ModelSim editor

- **Test environment design**

Writing test benches to ensure that the HDL description of a device is correct.

Tool: HDL Editor, Notepad++ & ModelSim editor

- **Behavioral simulation.**

This is an important stage that checks HDL correctness by comparing outputs of the HDL model and MATLAB fixed point model.

Tool: ModelSim[14].

- **Synthesis**

This process translates HDL code into a device netlist format, a complete circuit with logical elements for the design, Synthesis process checks code syntax and analyze the hierarchy of the design which ensures that the design is optimized for the design architecture.

Tool: ISE, Xilinx Synthesis Technology Tool (XST) [15].

- **Implementation**

This process consists of sequence of three steps

1. Translate: A process which combines all the input netlists and constraints to a logic design file. This information is saved as a NGD (Native Generic Database) file.

2. Map: A process which divides the whole circuit with logical elements into sub blocks such that they can be fit into the FPGA logic blocks, Configurable Logic Blocks (CLB) and Input Output Blocks (IOB), and generates an NCD (Native Circuit Description) file which physically represents the design mapped to the components of FPGA.

3. Place and Route: A process which places the sub blocks from the map process into logic blocks according to the constraints and connects the logic blocks. The PAR tool takes the mapped NCD file as input and produces a completely routed NCD file as output. Output NCD file contains the routing information.

Tool: ISE, Xilinx [15].

- **Device Programming**

This process loads the design on the FPGA. But the design must be converted to a format so that the FPGA can accept it. BITGEN program deals with the conversion. The routed NCD file is then given to the BITGEN program to generate a bit stream (a .BIT file) which can be used to configure the target FPGA device. This can be done using a cable.

Tool: ISE, Xilinx [15].

This Chapter discusses the above implementation steps for Chirp Spread Spectrum ZigBee Physical Layer transceiver. Also it presents performance and compare with floating point. Our procedure is to present each component using:

- 1- HDL entity
- 2- Input, Output and variable table which clarifies bit width and description.
- 3- Simple component architecture presents the main implementation idea.
- 4- Hardware implementation issue.

3.1 Chirp Spread Spectrum Physical Layer Transmitter

Section 2 presents the CSS PHY layer floating point model. We can notice that the source of the floating point is the chirp sequence generator so we only need to find the accepted analog to digital converter resolution for this block. Acceptance depends on Mean Square Error (MSE) between the fixed and floating point which should match the requirements is Section 6.5a.5.1.[8]. MSE can be calculating using the following equation:

$$meanSquareError = \left(\frac{\sum_N |exactValue - fixedValue|^2}{\sum_N |exactValue|^2} \right)$$

where:

exactValue: The floating point of chirp sequence samples multiplied by $2^{R-1} - 1$
As R is the analog to digital converter resolution

fixedValue: Round Value of *exactValue*

$N = 152$: The number of Samples in Chirp Sequence

We calculate the mean square Error for analog to digital converter resolution from 2 to 8. Result is shown in Figure 3-2.

According to [8], Section 6.5a.5.1, the accepted mean square Error is 0.005. we can observe from results that all $R > 4$ satisfy the required mean square Error

We choose $R = 5$ as it satisfy requirements and reduce the complexity and power consumptions.

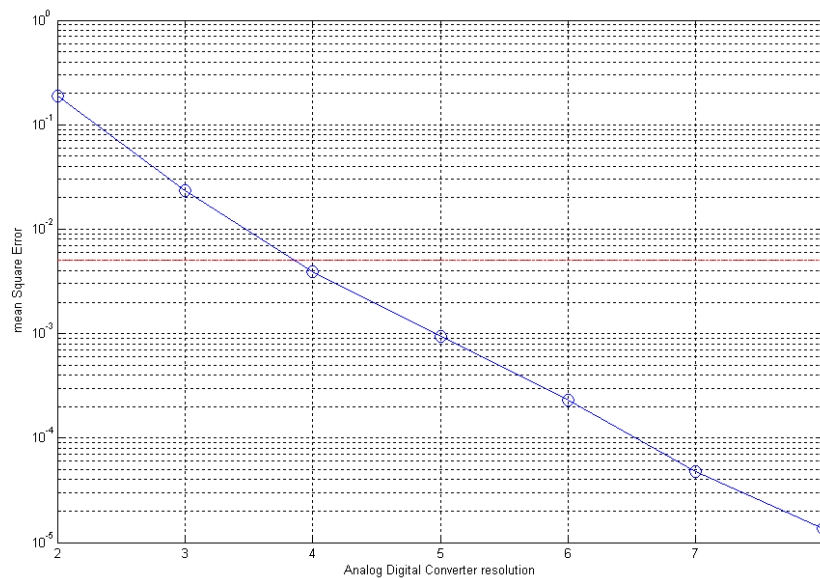


Figure 3-2, Analog Digital Converter resolution

➤ HDL Entity

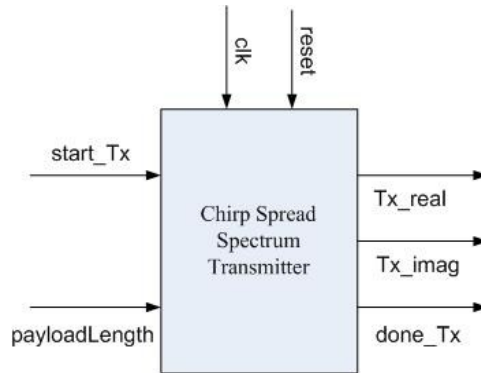


Table 3-1, CSS transmitter signal description

Signal	Width	Direction	Description
clk	1	IN	System clock
reset	1	IN	System reset
start_Tx	1	IN	Signal comes from MAC to PHY to initiate transmission process
PayloadLength	8	IN	Payload length in bytes
done_Tx	1	OUT	Signal goes from PHY to MAC to indicate finishing of transmission process
Tx_real	8	OUT	Real part of CSS Symbol
Tx_imag	8	OUT	Imaginary part of CSS Symbol

➤ **Architecture**

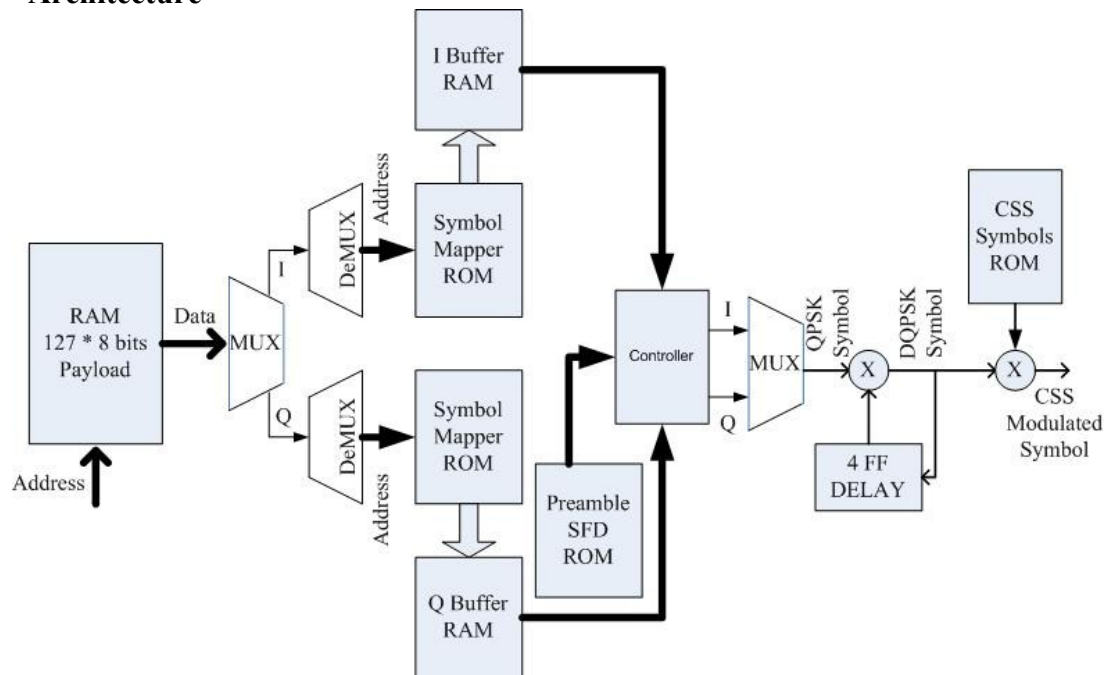


Figure 3-3, CSS transmitter architecture

➤ Hardware implementation

In this section our interest is implementation so we don't discuss all details about PHY processing, this has been discussed before, but we concentrate on digital design and architecture. **Figure 3-3** shows the architecture of CSS transmitter. The scenario of transmission goes as follows:

- MAC layer fills the payload RAM with payload data and sends the bytes number of payload to PHY.
 - MAC layer sends "start_Tx" signal in order to start transmission.
 - PHY layer starts reading the data from payload RAM and generate I & Q paths using multiplexers.
 - Symbols mapping is performed using ROMs, which save conversion tables, and demultiplexers.
 - Control Unit manage preamble, SFD, and I & Q Buffers data to form the packet
 - QPSK modulation is performed using multiplexers.
 - Differential QPSK is performed using 4 flip flops, to apply the delay, and multipliers.
 - CSS modulation is performed using ROM, which save chirp sequences as constants, and multipliers and some logic gates.
 - After finishing transmission PHY sends "done_Tx" signal to notify MAC layer.
- Hardware Verification

Verification aims to evaluate the design's compliance with its specified requirements. We use three different methods for verification, HDL test benches, "Electronic Design Automation" EDA Simulator Link and ChipScope. HDL test benches and EDA Simulator Link were used for unit and integration verification before device programming while ChipScope is used after. Hardware verification can be done at different stages of the process steps:

- **Behavioral Simulation (RTL Simulation)**

This is first of all simulation steps; those are encountered throughout the hierarchy of the design flow shown in Figure 3-1, Design flow. This simulation is performed before synthesis process to verify RTL (behavioral) code and to confirm that the design is functioning as intended. Behavioral simulation can be performed on either VHDL or Verilog designs. In this process, signals and variables are observed, procedures and functions are traced and breakpoints are set. This is a very fast simulation and so allows the designer to change the HDL code if the required functionality is not met with in a short time period. Since the design is not yet synthesized to gate level, timing and resource usage properties are still unknown.

- **Functional simulation (Post Translate Simulation)**

Functional simulation gives information about the logic operation of the circuit. Designer can verify the functionality of the design using this process after the Translate process. If the functionality is not as expected, then the designer has to make changes in the code and again follow the design flow steps.

- **Static Timing Analysis**

This can be done after synthesis and implementation processes. Post MAP timing report lists signal path delays of the design derived from the design logic. Post Place and Route timing report incorporates timing delay information to provide a comprehensive timing summary of the design.

3.1.1 HDL Testbenches

Testbenches are HDL program written for simulation, which is used to study the operation of a circuit or to verify the correctness of a design. Performing simulation is similar to doing an experiment with a physical circuit, in which we connect the circuit's input to a stimulus (e.g., a function generator) and observe the output (e.g., by logic analyzer). Simulating HDL description is like doing a virtual experiment, in which the physical circuit is replaced by the corresponding HDL description. Furthermore, we can develop HDL utility routines to imitate the stimulus generator (which is known as a test vector generator) and to collect and compare the output responses. This framework is known as a testbench.

Testbench consists of an entity declaration and architecture body. Since the testbench is self-contained, no port is specified in the entity declaration. There are three concurrent statements in the architecture body, including one component instantiation and two processes. The component instantiation specifies connections to the internal test generator and verifier. The first process is the stimulus generator. It produces all possible test vector combinations. The second process is the verifier. It takes the input test vector, waits the output settle down, checks the output value with the known value and reports the results. The two processes are for demonstration purposes only, and we don't need to worry about the syntax detail.



Figure 3-4, Testbench

Testbenches can be used for individual modules, Unit testing, or the whole system, Integration testing. The goal of unit testing is to isolate each part of the Design and show that the individual parts are correct. A unit test provides a strict, written contract that the piece of code must satisfy. As a result, it affords several benefits. Unit tests find problems early in the development cycle. The procedure is to write test cases for all modules so that whenever a change causes a fault, it can be quickly identified and fixed. By testing the parts of design first then testing the sum of its parts, integration testing becomes much easier.

In our work we have used testbenches to verify individual modules applying unit testing in some modules while other modules and whole design were verified using EDA Simulator Link

method.

3.1.2 EDA Simulator Link

EDA Simulator Link provides a bidirectional link between MATLAB and Simulink and MODELSIM, HDL simulators from Mentor Graphics, enabling verification of VHDL, Verilog, and mixed-language implementations.

EDA Simulator Link allows using MATLAB code or Simulink models as a testbench that generates stimulus for an HDL simulation and analyzes the simulation's response. Simply it replaces stimulus generator and verifier in testbench **Figure 3-** . It also lets you replace multiple HDL components with MATLAB code or Simulink models, enabling simulation of the complete system before all the HDL design elements are available.

EDA Simulator Link supports three workflows: MATLAB testbench, MATLAB component, and Simulink cosimulation. All workflows are shown in respectively.

➤ MATLAB testbench

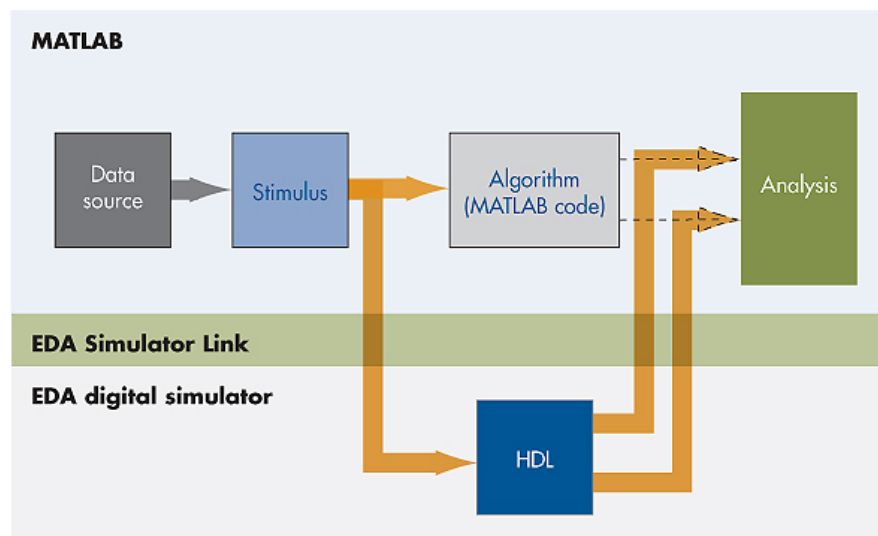


Figure 3-5, Using MATLAB code as a testbench supplying on-the-fly stimulus and analysis for an HDL simulation.

➤ MATLAB Component

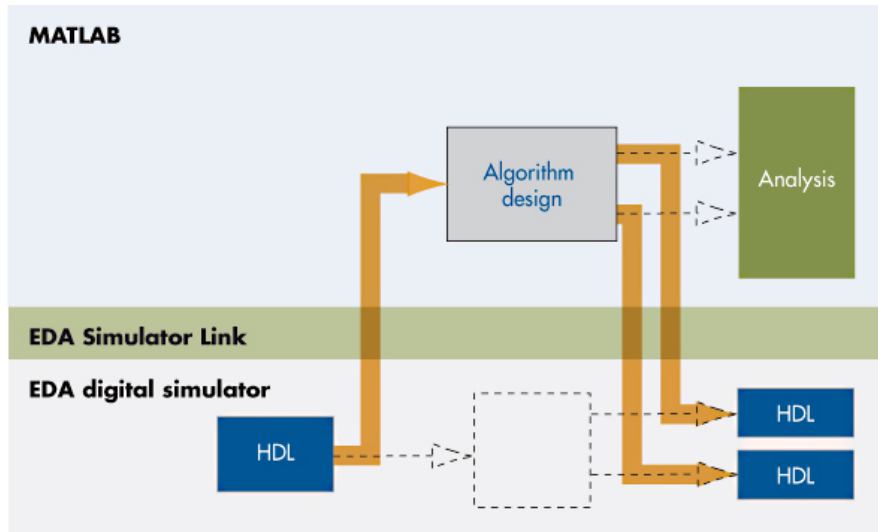


Figure 3-6, Substituting MATLAB code for an HDL component in an HDL simulation.

➤ Simulink Co-simulation

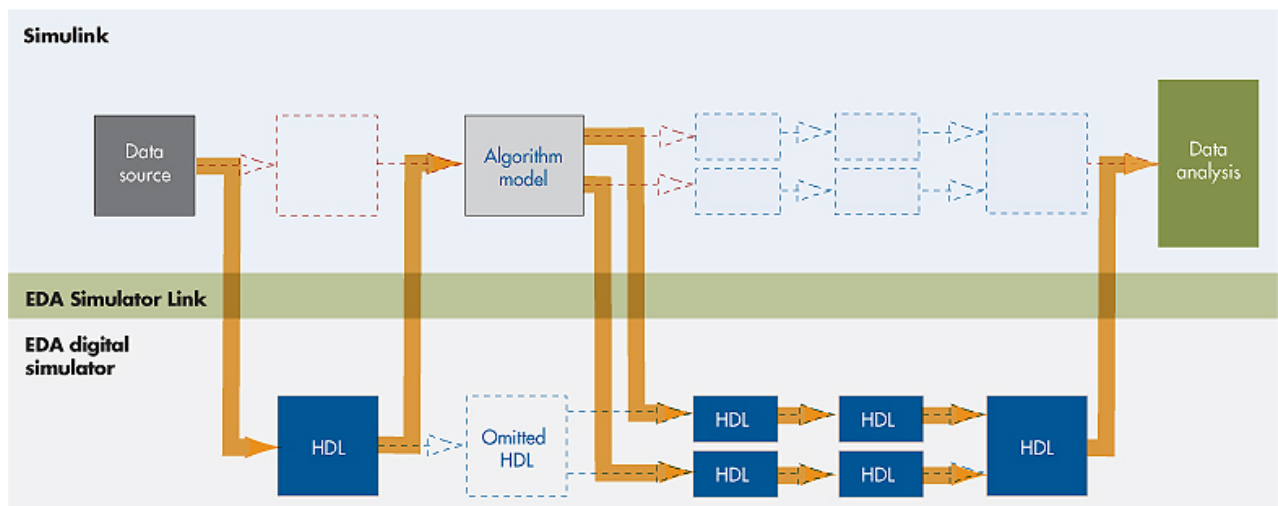


Figure 3-7, Mixing a Simulink system model with an HDL simulation as a bidirectional test bench and component.

Each workflow offers full interactive debugging capability, enabling simultaneous use of the full features of the MATLAB or Simulink and HDL simulator environments, including viewers, breakpoints, and pattern matching. EDA Simulator Link can interface with multiple HDL entities and multiple HDL simulators from a single MATLAB test bench or Simulink model. Multiple MATLAB test bench functions and Simulink cosimulation blocks can also be used with one or more HDL simulator.

Using EDA Simulator Link, you can set up an efficient environment for cosimulation,

component modeling, analysis, and visualization. In this environment, we can:

- Develop software test benches for HDL entities in MATLAB or Simulink
- Include HDL models in larger-scale system models developed and simulated in Simulink
- Verify, analyze, and visualize HDL implementation simulation results in MATLAB and Simulink
- Generate functional test vectors for manufacturing test programs and verification

Our work has interest in first workflow, MATLAB as a testbench **Figure 3-**. The steps to test a module Using MATLAB TestBench workflow as follows:

- Create the link between MATLAB and ModelSim, Linker script.
- Write the testbench in MATLAB.
- Generate test vectors using the fixed point module inputs.
- Run and add a break point in MATLAB after linker script to collect the output.
- Verify outputs by comparing them with fixed point module outputs.

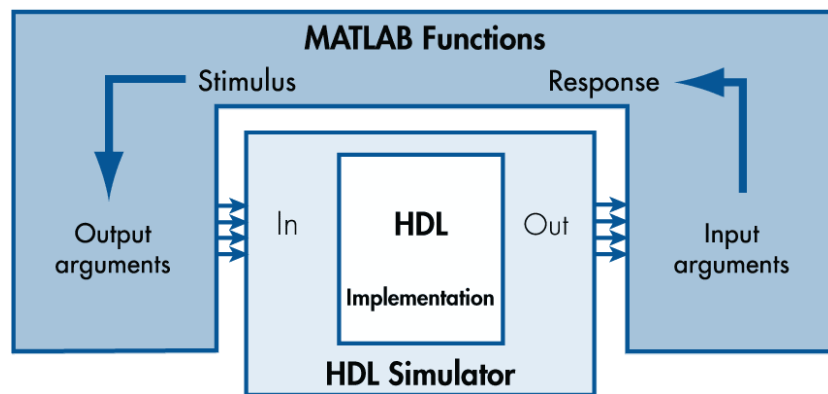


Figure 3-8, MATLAB code as a test bench for an HDL simulator

This method help us much to reduce the time of verification by debugging fast. It allows us to test any signal in the design module and compare it with fixed point reference. It also doesn't depend on file Input/Output interface, like in HDL testbenches to read test vectors and write outputs, which consume too much time. The Only drawback is using break points this is because MATLAB algorithms are untimed.

3.1.3 ChipScope (After Device Programming Verification)

ChipScope is a set of tools made by Xilinx that allows easily probe the internal signals of design inside an FPGA, much as using a logic analyzer. For example, while design is running on the FPGA, ChipScope allows triggering when certain events take place and view any of design's internal signals.

Because the ChipScope analyzer logic is implemented in the FPGA, it has some important limitations. The sample memory of the analyzer is limited by the memory resources of the FPGA. In a design that uses much of the FPGA's memory, there may not be much memory left over for the ChipScope cores. Also, ChipScope cannot sample as quickly as an external logic analyzer. Generally, ChipScope sampling rate will be the same as the design's clock frequency. It is therefore not possible to detect glitches.

In order to use the ChipScope internal logic analyzer in an existing design project, we first generate the ChipScope core modules, which perform the trigger and waveform capturing functionality on the FPGA. Afterwards, we instantiate these cores in our HDL code, and we connect those modules to the signals you want to monitor. The complete design is then recompiled. Instead of loading the resulting .bit file onto the FPGA using iMAPCT, the ChipScope Analyzer application is used to configure the FPGA. ChipScope Analyzer also provides the interface for setting the trigger criteria for the ChipScope cores, and for displaying the waveforms recorded by those cores.

REFERENCES

- [1] D. Estrin, L. Girod, G. Pottie, and M. Srivastava “Instrumenting the World with Wireless Sensor Networks,” Proc. Int’l Conf. Acoustics, Speech, and Signal Processing (ICASSP 2001), May 2001.
- [2] G.J. Pottie and W.J. Kaiser, “Wireless Integrated Network Sensors,” Communications of the ACM, vol. 43, no. 5, pp. 51-58, May 2000.
- [3] IEEE Std 802.15.4™-2003, Part 15.4: Wireless Medium Access Control (MAC) and Physical Layer (PHY) Specifications for Low-Rate Wireless Personal Area Networks (LR-WPANs), 2003.
- [4] IEEE Std 802.15.4a-2006, Wireless Medium Access Control (MAC) and Physical Layer (PHY) Specifications for Low-Rate Wireless Personal Area Networks (WPANs).
- [5] <http://www.zigbee.org>
- [6] <http://www.embedded.com>
- [7] J. Gutierrez, E. Callaway and R. Barret “Low-Rate Wireless Personal Area Networks.- Enabling Wireless Sensors with IEEE 802.15.4”, IEEE Press, 2003.
- [8] IEEE Std 802.15.4a-2007, Wireless Medium Access Control (MAC) and Physical Layer (PHY) Specifications for Low-Rate Wireless Personal Area Networks (WPANs), amendment 1: add alternate PHYs
- [9] Nanotron, “nanoNET Chirp Based Wireless Networks White Paper”, ver. 1.04, ID NA-04-0000-0298-1.04, February 2007.
- [10] <http://www.nanotron.com/EN/index.php>
- [11] John Proakis, “Digital communications”, MacGraw Hill, Third edition.
- [12] <http://www.xilinx.com/products/spartan3a/index.htm>
- [13] <http://www.mathworks.com/access/helpdesk/help/toolbox/fixedpoint/>
- [14] <http://model.com/>
- [15] <http://www.xilinx.com/tools/designtools.htm>
- [16] Jack E. Volder, “The CORDIC Trigonometric Computing Technique”, IRE Transactions on Electronic Computers, September 1959.

- [17] Ata Elahi & Adam Gschwender , “ZigBee Wireless Sensor and Control Network”, Prentice Hall , October 2009.
- [18] Simon Haykin & Barry Van Veen , “Signals and Systems” , John Wiley & Sons September 1998.
- [19] Bernard Sklar (Author), “Digital Communications: Fundamentals and Applications”, Prentice Hall; 2 edition, January 2001.
- [20] <http://www.doulos.com>
- [21] <http://www.vlsi-world.com/>
- [22] <http://www.asic-world.com/>
- [23] PONG P. CHU, “RTL HARDWARE DESIGN USING VHDL Coding for Efficiency, Portability, and Scalability ” , John Wiley & Sons, Inc., 2006.
- [24] PONG P. CHU, “FPGA Prototyping by VHDL Examples: Xilinx Spartan-3 Version” , Wiley, February 2008
- [25] Peter J Ashenden, “The Designer's Guide to VHDL” , Morgan Kaufmann, 2 edition, June 2001.
- [26] J. Bhasker Bk , “A Verilog HDL Primer”, Star Galaxy Press, February 2006
- [27] Steven W. Smith , “Digital Signal Processing: A Practical Guide for Engineers and Scientists”, <http://www.dspguide.com/> ,2002.
- [28] Janick Bergeron, “Writing Testbenches: Functional Verification of HDL Models” , Kluwer Academic, 2003.
- [29] IEEE Standard Verilog Hardware Description Language (IEEE Std 1364-2001), 2001.
- [30] IEEE Standard VHDL Language Reference Manual (IEEE Std 1076-2001), 2001.